Arjanveer Samra

Intro to Electrical Engineering

November 14, 2024

Lab Report #8

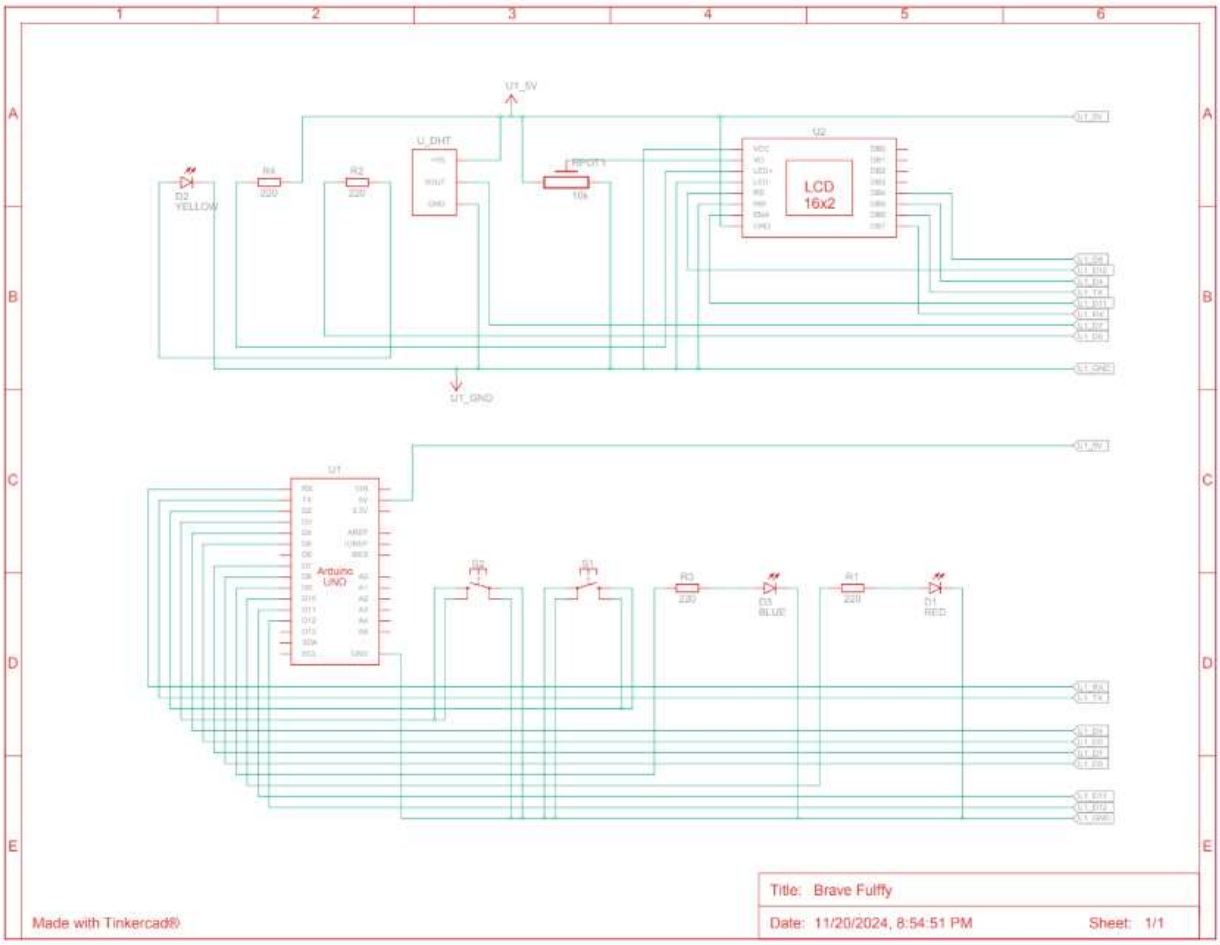
Overview

We are going to have an LCD display that reads the temperature and humidity of the room. Then off of that, we are going to implement more things to make it more sophisticated. For starters, after we get the temperature and humidity sensors to work we are going to connect 3 lights that light up depending upon our reading. The lights will represent a heater, a blower fan, and an air conditioner. Depending upon the value we input, will change what we need, and this will affect the button being pressed.

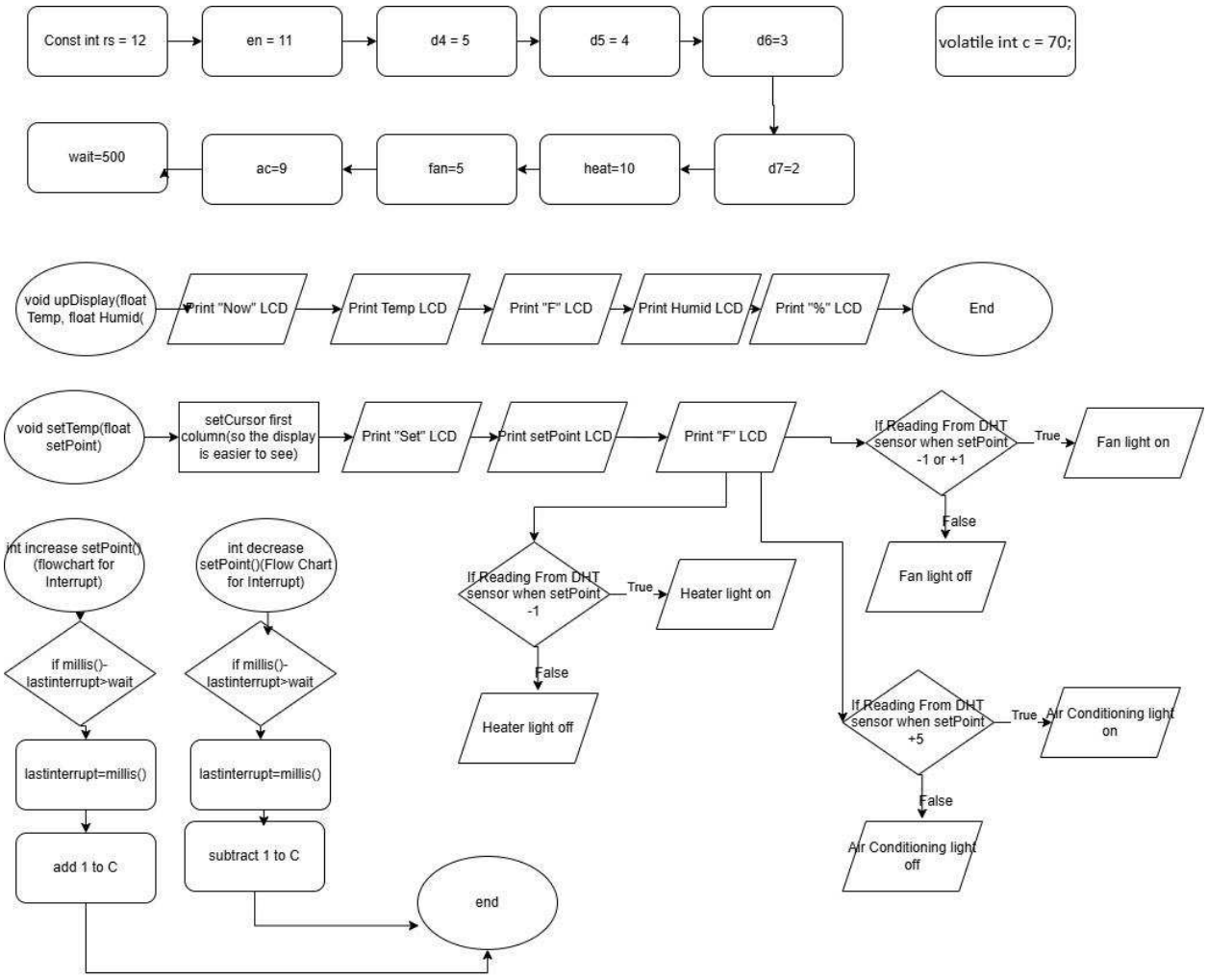
Requirements

I am going to have a value temperature set to default 70. But I'm also going to use two buttons that adjust this value to go up and down, due to the design verification since the room temp is going to be set at one value making it impossible to test. When the value is below its default of 70, the yellow light will blink. If it reaches all the way down to 65 and below the yellow and blue light will blink. On the other hand, if we push up to 72 and above the red and yellow light will blink. I will have to include and Interrupt for two of my button presses and they only work on certain digital pins, so I'll put them on pins 19 and 20. I will also have to create a function that accounts for the button bounce.

Wire Diagram



Software Diagram



Design Verification (Link to working video of the project -https://youtube.com)

```
#include <LiquidCrystal.h>
#include "DHT.h"
#define DHTPIN 7 // Connected to DHT sensor
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
// initialize the library by associating any needed LCD interface pin with the Arduino pin number it is connected to

const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2, Heater = 10, Fan = 8, AirConditoning = 9, wait = 500; //included Heater, Fan, and AirConditionin
volatile int c = 70;
// Tell the library which pins to use for each function
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);
//Records the last interrupt
unsigned long lastInterrupt = 0;



void setup() {
lcd.begin(16, 2);
dht.begin();
Serial.begin(9600);
//Starts the serial monitor and DHT sensor repsectively
pinMode(Heater, OUTPUT);
pinMode(Fan, OUTPUT);
pinMode(AirConditoning, OUTPUT);
pinMode(19, INPUT_PULLUP);
pinMode(20, INPUT_PULLUP);
//Heater Fan and AirConditioning have been intialized at the top and linked to LEDs, 19 and 20 are linked to te two buttons
attachInterrupt(digitalPinToInterrupt(20), increaseSetPoint, FALLING);
attachInterrupt(digitalPinToInterrupt(19), decreaseSetPoint, FALLING);
//the interrupt for each of the two buttons respectively
}
```

```
void Display(float Temp, int Humid) //displays the temp and humidity
{
  lcd.print("Now ");
  lcd.print(Temp);
  lcd.print("F ");
  lcd.print(Humid);
  lcd.print("%");
}



void setTemp(float setPoint)  //Makes LED light up based on tempertaure

{
  lcd.setCursor(0,1);
  lcd.print("Set ");
  lcd.print(setPoint);
  lcd.print("F");

  if(dht.readTemperature(true) < setPoint -1)
  {
    digitalWrite(Heater, HIGH);
  }
  else
  {
    digitalWrite(Heater, LOW);
  }

  if(dht.readTemperature(true) > setPoint +1 || dht.readTemperature(true) < setPoint -1)
  {
    digitalWrite(Fan, HIGH);
  }
  else
```

```
    else
    {
      digitalWrite(Fan, LOW);
    }

    if(dht.readTemperature(true) > setPoint +5 || dht.readTemperature(true) > setPoint +2 && dht.readHumidity() > 80)
    {
      digitalWrite(AirConditoning, HIGH);
    }
    else
    {
      digitalWrite(AirConditoning, LOW);
    }

  }

  void loop() {

    //displays the current temperature and humidity, and the set temperature
    Display( dht.readTemperature(true), dht.readHumidity());
    setTemp(c);
    delay(500);
    lcd.clear(); //clears the display allowing for the newer values of temp and humidity

    //prints all to serial montior
    Serial.print("Temperature");
    Serial.print(dht.readTemperature(true));
    Serial.print("°F ");
    Serial.print("Humidity: ");
    Serial.print(dht.readHumidity());
    Serial.println("%");
  }
```

```
int increaseSetPoint() //built in delay, since you can't have a delay on an interrupt, delay accounts for button bounce
{
  if(millis() - lastInterrupt > wait)
  {
    lastInterrupt = millis();
    c ++;
  }
}

//each function, decreaseSetPoint and increase SetPoint, run for a different button

int decreaseSetPoint()
{
  if(millis() - lastInterrupt > wait)
  {
    lastInterrupt = millis();

  c --;
  }
```

Learnings

I learned a lot about the project considering this was uniquely the most difficult and time-consuming part yet. I had to search with the help of a friend numerous things online when it came to the code. I had to look at YouTube tutorials to further learn and be able to code integrating the button bounce. Then I also needed something that accounted for the button bounce, and for that, I was able to search up and initialize “lastInterrupt”. Overall this lab pushed me the most and forced me to self-study and learn a lot more about Arduino code.

Conclusion

The project all in all went well as I was able to complete what was asked. But one thing that didn't was the time I used. Since in the real world things like this need to be made and pushed out a lot faster. Also, one more thing I could have done is read the Lab Procedure more in-depth. I say this a lot but the more I read the more I realise just how important it truly is. It might seem boring or like busy work at first glance, but it goes a long way toward completing your project.